SEMANA 3 · UNIDAD 3

Modelos de Aprendizaje No Supervisado

3 – 16 oct 2026 (no hay tutoría el 10/10)

Clase sincrónica: sábado 03/10 · tutoría 3 h + laboratorio 1.5 h

48 h: 4.5 ACD · 18 APE · 25.5 AA

RDA2 CG-G2 CE-G2 Bibliografía: BB1 · BB3 · BB4 · BB5 · BB7

MANUAL M3 · TEORÍA DE LA SEMANA

Clustering, reducción de dimensionalidad y anomalías, con mirada ética

Lectura base de la semana 3 (2.5–3 h). Cada sección termina con un enlace **Practica esto** → al nivel exacto del ejercicio o a la animación que la ilustra. Las cifras citadas salen de la ejecución real de los notebooks E3.1–E3.4.

CONTENIDO

§1 Aprender sin etiquetas
§2 K-means
§3 Escalamiento y distancias
§4 DBSCAN
§5 Validación de clusters
§6 PCA
§7 t-SNE
§8 Detección de anomalías
§9 Ética de lo no supervisado (RDA2)
§10 Autoevaluación y lecturas

§1 Aprender sin etiquetas

En el aprendizaje no supervisado solo hay variables X, sin una respuesta y que diga qué es correcto. Las tareas de esta semana son tres:

- **Clustering:** encontrar grupos de observaciones parecidas (segmentos de clientes, zonas geográficas, tipos de conexión de red).
- **Reducción de dimensionalidad:** resumir muchas variables en pocas, para comprimir, quitar ruido, visualizar o preparar un modelo supervisado.
- **Detección de anomalías:** encontrar las observaciones que no se parecen al resto (fraude, intrusiones, fallas, errores de captura).

Sin etiquetas no hay exactitud que calcular. Por eso la validación se apoya en tres fuentes:

- **Validación interna:** criterios geométricos calculados con los mismos datos (inercia, silhouette, Davies-Bouldin, varianza explicada).
- **Estabilidad:** el resultado no debería cambiar mucho con otra semilla u otra muestra (ARI entre bootstraps).
- **Validación externa y utilidad:** información que no participó en el algoritmo (una etiqueta guardada aparte, la opinión de expertos o una métrica de negocio) y la pregunta de si el resultado sirve para decidir algo. En E3.1 la respuesta a la campaña no se usó para segmentar, pero su tasa va de 5.9 % a 23.1 % entre segmentos: la segmentación captura algo relevante.

> ⚠ **El algoritmo siempre encuentra algo**
> K-means devuelve k grupos aunque los datos sean ruido uniforme, PCA devuelve componentes aunque no haya estructura y un detector de anomalías siempre marca a alguien. El resultado depende además de la escala de las variables, de los parámetros y de la semilla. Una solución no supervisada es una **hipótesis** que hay que validar, no un hallazgo.

Practica esto → **E3.1 · Nivel 3** (validación externa con la tasa de contratación) · **Animación A3.1**

§2 K-means

K-means busca k centroides $\mu_1, \ldots, \mu_k$ que minimicen la **inercia**, la suma de distancias al cuadrado de cada punto a su centroide:

$$J = \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2$$

El algoritmo de **Lloyd** alterna dos pasos hasta que las asignaciones dejan de cambiar:

1. **Asignar:** cada punto va al centroide más cercano.
2. **Actualizar:** cada centroide pasa a ser la media de sus puntos.

Cada paso baja o mantiene J, así que el algoritmo converge, pero a un **óptimo local** que depende de dónde empiezan los centroides. La inicialización **k-means++** elige el primer centro al azar y cada nuevo centro con probabilidad proporcional a $D(x)^2$, la distancia al cuadrado al centro más cercano ya elegido: los centros quedan repartidos. En E3.1 (5 grupos, 30 corridas), el inicio aleatorio llegó al mejor óptimo ($J = 1035.6$) en el 63 % de las corridas y k-means++ en el 90 %; la peor corrida aleatoria quedó en $J = 2295.7$. Por eso se combinan k-means++ y varias inicializaciones (`n_init`) y se queda la de menor inercia.

Elegir k

- **Codo:** la inercia siempre baja al aumentar k (con $k = n$ vale cero), así que se busca dónde deja de bajar mucho. Muchas veces el codo no es nítido.
- **Silhouette y Davies-Bouldin** (§5): en los datos sintéticos de E3.1, k = 5 (el verdadero) tiene la mayor silhouette (0.612) y el menor Davies-Bouldin (0.516).
- **Estabilidad:** con k = 5 el ARI medio entre bootstraps es 0.988; con k = 8, 0.769: cuando k es incorrecto, el algoritmo parte los grupos de una forma distinta en cada muestra.
- **Utilidad:** un k que se puede describir y usar vale más que un k que gana por milésimas en un índice.

> **Qué supone K-means**
> Grupos **convexos** y más o menos esféricos, de tamaños y varianzas parecidas, en un espacio donde la distancia euclidiana tiene sentido. Con dos lunas entrelazadas las parte por la mitad (ARI 0.25 en E3.2) y con grupos alargados corta en el lugar equivocado. Todo punto recibe un grupo: no hay ruido. Para millones de filas existe `MiniBatchKMeans`.

Python

```python
from sklearn.cluster import KMeans
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

modelo = make_pipeline(StandardScaler(),
                       KMeans(n_clusters=5, n_init=10, random_state=2026))
etiquetas = modelo.fit_predict(X)
inercia = modelo[-1].inertia_          # J de la mejor de las 10 inicializaciones
```

> **Practica esto →** **E3.1 · Niveles 1 y 2**
> (Lloyd y k-means++ desde cero; tu inercia debe coincidir con `KMeans`) · **Animación A3.1**

§3 Escalamiento y distancias

K-means, DBSCAN, LOF, kNN y PCA dependen de distancias o varianzas. Si el ingreso está en dólares (desviación de decenas de miles) y la edad en años (desviación de una decena), la distancia euclidiana es casi solo ingreso: la edad no influye en quién está cerca de quién. Antes de agrupar hay que decidir cuánto pesa cada variable, y la escala es esa decisión.

- **Estandarización** (`StandardScaler`): media 0 y desviación 1. Es la opción por defecto, pero la media y la desviación son sensibles a los atípicos.

- **Escalado robusto** (`RobustScaler`): resta la mediana y divide por el rango intercuartílico. Los atípicos no deforman la escala de los demás puntos, aunque siguen siendo extremos.
- **Transformaciones de forma:** logaritmo para montos con cola larga; `QuantileTransformer` para llevar cada variable a rangos uniformes. En E3.1 el saldo bancario va de -8 019 a 102 127 con mediana 448: se usa la transformación por cuantiles para que unos pocos saldos enormes no formen segmentos propios.
- **Variables categóricas:** la codificación one-hot hace que cada categoría pese como una variable entera y las distancias entre categorías son todas iguales. Hay alternativas fuera de scikit-learn (distancia de Gower, K-prototypes); la opción más simple es segmentar solo con variables numéricas y usar las categóricas para describir los grupos.

En la animación A3.2, sin escalar, K-means agrupa solo por ingreso. Al estandarizar recupera los tres grupos reales. Si se agregan unos pocos ingresos extremos, con k = 3 uno de los centroides se va a los atípicos y dos grupos reales se fusionan; con escalado robusto y un cuarto grupo, los atípicos quedan aislados y los tres grupos reales se recuperan.

> ⚠ **Errores frecuentes con la escala**
> Agrupar variables en unidades distintas sin escalar; estandarizar variables muy asimétricas y concluir que "los clusters son los clientes ricos"; incluir identificadores o códigos numéricos (código postal, número de cliente) como si fueran magnitudes; escalar con toda la base cuando luego se asignarán clientes nuevos (el escalador se ajusta una vez y se guarda con el modelo, dentro del pipeline).

Practica esto → **E3.1 · Nivel 3** (transformación por cuantiles del saldo y los contactos) · **Animación A3.2**

§4 DBSCAN

DBSCAN define un cluster como una **región densa**. Tiene dos parámetros: el radio ε y el mínimo de puntos `minPts` (`min_samples` en scikit-learn, que cuenta al propio punto).

- La **ε-vecindad** de p es $N_\varepsilon(p) = \{q : d(p,q) \le \varepsilon\}$.
- p es **núcleo** si $|N_\varepsilon(p)| \ge \text{minPts}$; es **borde** si no es núcleo pero está en la vecindad de un núcleo; si no, es **ruido** (etiqueta -1).
- q es **alcanzable por densidad** desde p si hay una cadena de núcleos, cada uno en la vecindad del anterior, que lleva de p a q. Un cluster es un conjunto máximo de puntos conectados por densidad y se construye por expansión en anchura desde un núcleo: los bordes se agregan pero no expanden.

DBSCAN no necesita saber cuántos grupos hay, encuentra formas arbitrarias y separa el ruido. En las lunas con ruido de E3.2 (eps = 0.15, minPts = 6) logra ARI 0.990 frente a 0.248 de K-means y deja 13 puntos como ruido.

Elegir eps: el gráfico de k-distancia

Para cada punto se calcula la distancia a su k-ésimo vecino (k = minPts, contando al propio punto), se ordenan y se grafican. Los puntos de zonas densas tienen distancias pequeñas; el ruido, grandes. El **codo** de la curva es un buen candidato para eps (en E3.2, el percentil 90 de la 6-distancia es 0.145 y el 95, 0.186). Para minPts, una regla habitual es al menos la dimensión más uno, y el doble de la dimensión con datos ruidosos.

> **Coordenadas geográficas: haversine**
> La distancia euclidiana entre grados de latitud y longitud no es una distancia sobre la Tierra (un grado de longitud mide menos cerca de los polos). Con `metric="haversine"`, scikit-learn espera **[latitud, longitud] en radianes** y un eps también en radianes: $\varepsilon = \mathrm{km}/6371$. En California (E3.2, eps = 5 km, minPts = 60) aparecen 22 zonas densas, que coinciden con las áreas metropolitanas, y el 29.7 % de los distritos queda como ruido: zonas poco pobladas, no errores.

Python

```python
import numpy as np
from sklearn.cluster import DBSCAN

coords = np.radians(df[["Latitude", "Longitude"]].to_numpy())
db = DBSCAN(eps=5 / 6371, min_samples=60, metric="haversine",
            algorithm="ball_tree").fit(coords)
zonas = db.labels_                      # -1 = ruido
print(zonas.max() + 1, "zonas ·", np.mean(zonas == -1), "de ruido")
```

Límites. Un solo eps no sirve cuando los grupos tienen **densidades distintas**: el eps que separa el grupo denso fragmenta el disperso, y el que une el disperso fusiona todo (animación A3.3). En alta dimensión las distancias se parecen entre sí y la densidad pierde sentido. El resultado es muy sensible a los parámetros: en California, eps = 3 km con minPts = 120 deja el 87 % como ruido, y eps = 20 km con minPts = 30, solo el 3 %. `HDBSCAN` (en scikit-learn desde la versión 1.3) explora muchos eps a la vez y tolera densidades distintas.

> **Practica esto → E3.2 · Niveles 1 a 4**
> (DBSCAN desde cero, k-distancia, haversine y sensibilidad) · **Animación A3.3**

§5 Validación de clusters

La **silhouette** mide, punto por punto, si está más cerca de su grupo que del grupo vecino. Con $a(i)$ la distancia media de i a los demás puntos de su cluster y $b(i)$ la menor distancia media de i a los puntos de otro cluster:

$$s(i) = \frac{b(i) - a(i)}{\max\big(a(i),\, b(i)\big)} \in [-1, 1]$$

Cerca de 1, el punto está bien ubicado; cerca de 0, está en la frontera; negativa, está más cerca del grupo vecino que del suyo (mal asignado). La media de $s(i)$ resume la partición y el **gráfico de silhouette** (las barras de cada grupo ordenadas) muestra qué grupos son débiles. Su costo es cuadrático en n: con muchos datos se calcula sobre una muestra (`sample_size`).

- **Davies-Bouldin:** para cada grupo, la peor razón entre la suma de las dispersiones de dos grupos y la distancia entre sus centroides; se promedia. **Más bajo es mejor.**
- **Estabilidad:** se repite el clustering con otras semillas o en muestras bootstrap y se compara con el **índice de Rand ajustado** (ARI): 1 = idénticas, ≈ 0 = coincidencia al azar. El ARI no depende de cómo se numeren los grupos.
- **Perfiles:** medianas y proporciones de cada variable por grupo, y una frase que describa a cada segmento. Si no se puede describir, no se puede usar.

> **Con datos reales, los índices no eligen por ti**
> En Bank Marketing (E3.1, seis variables de comportamiento), la silhouette sube hasta k = 8 (0.486) porque las variables binarias crean grupos discretos, y Davies-Bouldin mejora poco a poco. Se eligió k = 4 porque es estable (ARI 0.88 entre bootstraps) y útil: "con hipoteca, nunca contactados" (36 %), "sin hipoteca ni préstamo" (32 %), "contactados en campañas anteriores" (18 %, contratación 23 %) y "con préstamo personal" (14 %, contratación 6 %). Las métricas internas premian la geometría; la decisión combina geometría, estabilidad y uso.

Python

```python
from sklearn.metrics import adjusted_rand_score, davies_bouldin_score, silhouette_score

s = silhouette_score(Z, etiquetas, sample_size=8000, random_state=2026)
db = davies_bouldin_score(Z, etiquetas)
ari = adjusted_rand_score(etiquetas_semilla_1, etiquetas_semilla_2)   # estabilidad
```

Practica esto → **E3.1 · Niveles 2 y 3** (silhouette, Davies-Bouldin, ARI y perfiles) · **Animación A3.4** (haz clic en un punto para ver a(i) y b(i))

§6 PCA

El análisis de componentes principales busca las direcciones en las que los datos **varían más**. Con los datos centrados X_c $(n \times p)$ y su matriz de covarianza $\Sigma = X_c^\top X_c/(n-1)$:

- la primera componente es el vector unitario v_1 que maximiza la varianza de la proyección $X_c v_1$; es el vector propio de Σ con el mayor valor propio λ_1. Las siguientes hacen lo mismo, ortogonales a las anteriores;
- la **varianza explicada** por la componente j es λ_j, y su proporción, $\lambda_j / \sum_l \lambda_l$;
- en la práctica se usa la descomposición en valores singulares $X_c = U \Sigma_s V^\top$: las columnas de V son las componentes y $\lambda_j = \sigma_j^2/(n-1)$.

$$Z = X_c V_k \quad \text{(proyección)}, \qquad \hat{X} = Z V_k^\top + \bar{x} \quad \text{(reconstrucción)}$$

Maximizar la varianza proyectada equivale a minimizar el error de reconstrucción. En la nube 3D de E3.3 las proporciones son 85.6 %, 13.9 % y 0.6 %: con k = 2 el error medio de reconstrucción es 0.061, frente a 1.565 con k = 1. En los 400 rostros de Olivetti (4 096 píxeles), 50 componentes explican el 87.4 % de la varianza y la cara ya se reconoce (las **eigenfaces**).

Cuántas componentes

- **Varianza acumulada** (p. ej. 90–95 %; `PCA(n_components=0.95)` elige k solo). En digits, 2 componentes explican 28.5 % y hacen falta 21 para el 90 %.
- **Codo** del gráfico de sedimentación (scree plot).
- **Desempeño del modelo que viene después**, por validación cruzada: en digits, una regresión logística logra 0.971 con los 64 píxeles, 0.955 con 30 componentes y 0.815 con 5.

> ⚠ **PCA sin escalar sigue a la variable más grande**
>
> La varianza depende de las unidades: si una variable está en miles, la primera componente será casi esa variable (animación A3.5, escena de la escala). Estandariza antes, salvo que todas las variables estén en la misma unidad (píxeles). Y como con cualquier transformación aprendida, PCA va **dentro del pipeline**: se ajusta solo con los folds de entrenamiento. Las componentes son combinaciones de todas las variables; interpretarlas exige mirar sus cargas (`components_`) y aun así es difícil.

```python
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

pca = PCA().fit(StandardScaler().fit_transform(X))
acumulada = pca.explained_variance_ratio_.cumsum()

pipe = make_pipeline(StandardScaler(), PCA(n_components=30),
                     LogisticRegression(max_iter=3000))
print(cross_val_score(pipe, X, y, cv=5).mean())
```

Practica esto → **E3.3 · Niveles 1 a 3** (PCA por SVD, eigenfaces y PCA en un pipeline) · **Animación A3.5**

§7 t-SNE

t-SNE es una técnica **no lineal** para visualizar datos de muchas dimensiones en 2D intentando que los vecinos sigan siendo vecinos. Funciona en dos pasos:

1. En el espacio original, convierte distancias en **probabilidades de ser vecinos** con una gaussiana centrada en cada punto: $p_{j|i} \propto \exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)$. El ancho σ_i se ajusta para que la distribución tenga una **perplejidad** dada ($2^{H(P_i)}$, el número efectivo de vecinos). Luego simetriza: $p_{ij} = (p_{j|i} + p_{i|j})/2n$.
2. En el mapa 2D usa una distribución t de Student de colas pesadas, $q_{ij} \propto (1 + \|y_i - y_j\|^2)^{-1}$, y mueve los puntos por descenso de gradiente para minimizar la divergencia $\mathrm{KL}(P \,\|\, Q) = \sum_{i \neq j} p_{ij} \log(p_{ij}/q_{ij})$. Las primeras iteraciones exageran las atracciones (×12) para que se formen los grupos.

La KL castiga mucho poner lejos a dos vecinos cercanos y poco lo contrario: t-SNE conserva los **vecindarios locales** y sacrifica la geometría global. En la animación A3.6 (360 dígitos, perplejidad 30), el 69 % de los 10 vecinos de cada dígito siguen entre sus 10 vecinos en el mapa y el 96 % de los dígitos tiene como vecino más cercano otro de su misma clase. En E3.3, con los 1 797 dígitos, PCA a 2D (28.5 % de la varianza) mezcla varias clases y t-SNE las separa en islas.

> ⚠ **Lo que un mapa de t-SNE no dice**
>
> En E3.3, un grupo 9.5 veces más disperso que otro aparece solo 1.4 veces más ancho en el mapa, y un grupo 17 veces más lejano aparece apenas 2.8 veces más lejos (PCA conserva 9.8 y 17.2). En t-SNE **no se leen** los tamaños de los grupos, las distancias entre grupos, la posición absoluta ni la densidad. Con perplejidad muy baja aparecen **subgrupos que no existen**. Y otra semilla da un mapa rotado y reordenado.

- **Buenas prácticas:** mostrar al menos dos perplejidades (5–50; el valor por defecto es 30) y más de una semilla; `init="pca"` (por defecto en scikit-learn); estandarizar antes; con muchas variables, reducir primero con PCA a unas 50 componentes.
- **Solo para explorar:** `TSNE` no tiene `transform` para datos nuevos y sus coordenadas cambian con los parámetros. No se usan como variables de un modelo ni para medir "cuán distintos" son dos segmentos: esas afirmaciones se verifican en el espacio original.

```python
from sklearn.manifold import TSNE

# siempre varias perplejidades
mapas = {p: TSNE(n_components=2, perplexity=p, init="pca",
                 random_state=2026).fit_transform(X_escalado)
         for p in (5, 30, 50)}
```

> **Practica esto →** **E3.3 · Niveles 3 y 4**
> (PCA frente a t-SNE en digits y el error de interpretación típico) · **Animación A3.6**
> (t-SNE en vivo)

§8 Detección de anomalías

Una anomalía es una observación que no se parece al resto. Conviene distinguir la **detección de atípicos** (los datos de entrenamiento ya incluyen anomalías: el detector debe aislarlas) de la **detección de novedades** (se entrena con datos verificados como normales y se marca lo nuevo que se aparta). Las anomalías pueden ser puntuales, contextuales (normal en diciembre, raro en marzo) o colectivas (una secuencia rara de eventos normales).

Reglas estadísticas

- **z-score:** marcar si $|x - \bar{x}|/s > 3$. Supone normalidad y sufre **enmascaramiento**: los propios atípicos inflan $\bar{x}$ y s. En E3.4 (1 000 tiempos de respuesta, 50 anómalos) detecta solo 33 de 50.
- **IQR (Tukey):** marcar fuera de $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$. Es robusta, pero con variables asimétricas marca la cola normal: detecta 50 de 50 con 25 falsas alarmas; sobre $\log x$, 45 de 50 con 4.
- Las reglas univariadas no ven anomalías que solo aparecen en la **combinación** de variables.

Isolation Forest

Idea: una anomalía es **fácil de aislar**. Un árbol de aislamiento elige una variable al azar y un umbral al azar entre su mínimo y su máximo, y repite en cada lado. La **longitud del camino** $h(x)$

(número de cortes hasta dejar a x solo) es corta para los puntos raros. Se promedian muchos árboles, cada uno sobre una submuestra de tamaño ψ (256 por defecto), y se normaliza:

$$s(x) = 2^{-E[h(x)]/c(\psi)}, \qquad c(\psi) = 2H(\psi - 1) - \frac{2(\psi - 1)}{\psi}$$

donde H es el número armónico y $c(\psi)$ el camino medio esperado. Si s está cerca de 1, el punto es anómalo; alrededor de 0.5 o menos, normal. En la animación A3.7, una anomalía se aísla en 3 cortes y un punto del centro de la nube en 14; con 100 árboles, 3.6 frente a 11.9 cortes en promedio ($s = 0.75$ frente a 0.39). Es rápido, escala bien y no necesita escalar las variables.

LOF: densidad local

El *Local Outlier Factor* compara la densidad de un punto con la de sus vecinos. Con $N_k(p)$ sus k vecinos más cercanos y $\text{k-dist}(o)$ la distancia de o a su k-ésimo vecino:

$$\begin{aligned} \text{alc}_k(p, o) &= \max\{\text{k-dist}(o),\ d(p, o)\} \\ \text{lrd}(p) &= \Big(\frac{1}{k} \sum_{o \in N_k(p)} \text{alc}_k(p, o)\Big)^{-1} \\ \text{LOF}(p) &= \frac{1}{k} \sum_{o \in N_k(p)} \frac{\text{lrd}(o)}{\text{lrd}(p)} \end{aligned}$$

Un LOF cercano a 1 indica un punto tan denso como su vecindario; claramente mayor que 1, un punto **anómalo respecto de su vecindario**. Así detecta atípicos locales junto a grupos densos que una regla global no ve: en la animación A3.8, un punto aislado junto a un grupo denso tiene LOF 3.62 (puesto 2 de 172), pero por distancia a su k-ésimo vecino queda en el puesto 14, detrás de puntos normales de un grupo disperso. `n_neighbors` (20 por defecto) fija la escala del vecindario. Para puntuar datos nuevos se usa `novelty=True`.

El umbral y la evaluación

Los detectores producen un **puntaje**. `contamination` solo fija qué fracción marcar (el umbral de `predict`); no cambia el orden. En scikit-learn, `score_samples` devuelve el negativo del puntaje (más bajo = más anómalo). Para evaluar:

- **Con etiquetas** (aunque sean pocas): PR-AUC, cuya línea base es la prevalencia; recall con una tasa de falsas alarmas fija; qué **tipos** de anomalía se detectan.
- **Sin etiquetas:** revisión experta de los primeros puntajes, estabilidad entre semillas y muestras, inyección de anomalías sintéticas y comparación con un baseline sencillo.

> **Lo que muestra KDD Cup 99 (E3.4)**
> 100 655 conexiones de red, 3.4 % ataques. Entrenando con datos contaminados, ningún método detecta `smurf`, más del 70 % de los ataques: son miles de conexiones casi idénticas que forman una región **densa** y pasan por normales (enmascaramiento). Entrenando solo con tráfico normal (detección de novedades), el baseline más simple (el |z| más extremo de cada conexión) logra PR-AUC 0.906, Isolation Forest 0.714 y LOF 0.553. Para detectar el 80 % de los ataques, Isolation Forest genera 175 falsas alarmas por cada 10 000 conexiones normales y el baseline, 24. Los métodos no supervisados detectan lo **raro**, no necesariamente lo **malicioso**, y siempre se comparan con un baseline.

El umbral operativo lo fija la **capacidad de revisión**: si el equipo investiga 300 alertas por cada 10 000 conexiones, se marcan los 300 puntajes más altos; en E3.4 eso detecta el 82 % de los ataques con una precisión de 0.92. Las falsas alarmas cuestan horas de analistas y producen fatiga de alertas.

```python
from sklearn.ensemble import IsolationForest
from sklearn.metrics import average_precision_score
from sklearn.neighbors import LocalOutlierFactor

iso = IsolationForest(n_estimators=200, random_state=2026).fit(X_normal)
puntaje_if = -iso.score_samples(X_nuevo)              # mayor = más anómalo

lof = LocalOutlierFactor(n_neighbors=35, novelty=True).fit(Z_normal)
puntaje_lof = -lof.score_samples(Z_nuevo)

print(average_precision_score(y_nuevo, puntaje_if))  # solo si hay etiquetas
alertas = np.argsort(-puntaje_if)[:300]               # umbral = capacidad del equipo
```

> **Practica esto → E3.4 · Niveles 1 a 4**
> (reglas, Isolation Forest de juguete, KDD Cup 99 y umbral por capacidad) ·
> **A3.7 Isolation Forest · A3.8 LOF**

§9 Ética de lo no supervisado (RDA2)

Segmentar o marcar como anómalas a **personas** tiene consecuencias: a quién se ofrece un producto, a quién se llama, a quién se investiga o se bloquea. Que no haya etiquetas no significa que no haya decisiones.

- **Proxies:** quitar las variables sensibles no vuelve neutral a una segmentación. En E3.1 la edad no se usó para segmentar, pero la razón de representación de los mayores de 60 años va de 0.26 a 1.74 entre segmentos: la hipoteca, el saldo o haber sido contactado antes se relacionan con la etapa de vida. Lo mismo ocurre con zonas geográficas y nivel socioeconómico u origen (E3.2).
- **Las variables sensibles se conservan para auditar,** aunque no entren al modelo: sin ellas no se podría detectar el desbalance.
- **Estigmatización:** los nombres de los segmentos ("riesgosos", "de bajo valor") y las zonas marcadas pueden reforzar prejuicios y reproducir exclusiones históricas, como el *redlining* en créditos y seguros.
- **Impacto desigual de las anomalías:** los grupos minoritarios o con comportamiento distinto (clientes nuevos, otra región, turnos nocturnos) son "raros" por definición y pueden recibir más alertas sin tener más riesgo. Hay que medir las alertas y las falsas alarmas **por grupo**.
- **Uso previsto:** el mismo resultado puede ser aceptable para priorizar llamadas y problemático para fijar tasas o negar un servicio. La revisión humana y la posibilidad de apelar son parte del diseño.

Análisis por subgrupos (lo que pide la Tarea 3)

Para cada cluster (y para el conjunto marcado como anómalo): la proporción de cada grupo (sexo, educación, edad) dentro del cluster y la **razón de representación** = proporción en el cluster / proporción en la población. Para las anomalías, la **tasa de marcado** de cada grupo y su razón frente a la del grupo de referencia. Una banda de referencia como 0.8–1.25 (inspirada en la regla de los cuatro quintos) sirve para priorizar qué revisar; no es un criterio legal ni reemplaza la pregunta de para qué se usará el resultado. Documenta los hallazgos, las limitaciones y los usos que no recomiendas.

Python

```python
import pandas as pd

composicion = pd.crosstab(df["segmento"], df["grupo_edad"], normalize="index")
# razón de representación: 1 = igual que en la población
razon = composicion / df["grupo_edad"].value_counts(normalize=True)
tasa_alerta = df.groupby("sexo")["alerta"].mean()
razon_alerta = tasa_alerta / tasa_alerta.min()
```

Practica esto → **E3.1 · Nivel 4** (¿la segmentación discrimina?) · **Tarea 3** (análisis por subgrupos, 25 de 100 puntos)

§10 Autoevaluación y lecturas

▼ 1. Corres K-means dos veces con k = 4 y obtienes inercias 1 836 y 2 410. ¿Qué pasó y qué haces?

La segunda corrida quedó en un óptimo local peor, por la inicialización. Usa k-means++ con varias inicializaciones (`n_init=10`) y quédate con la de menor inercia; revisa también la estabilidad entre semillas.

▼ 2. La silhouette media de tu segmentación es 0.47 y un grupo tiene muchas silhouettes negativas. ¿Qué significa?

Esos puntos están, en promedio, más cerca de otro grupo que del suyo: el grupo es débil o está mal definido. Prueba otro k, revisa la escala y mira el perfil de ese grupo antes de usarlo.

▼ 3. Agrupas tiendas por latitud y longitud con DBSCAN y eps = 0.05. ¿Qué está mal?

Si usas distancia euclidiana, 0.05 grados no es una distancia fija sobre la Tierra. Usa `metric="haversine"` con coordenadas en radianes y eps = km / 6371, y elige los km según la pregunta (barrio, ciudad o región).

▼ 4. Un informe dice: "en el mapa de t-SNE el segmento A está muy lejos del B, así que son muy distintos". ¿Es válido?

No. t-SNE no conserva las distancias entre grupos (en E3.3 una razón de 17 quedó en 2.8). Esa afirmación se comprueba en el espacio original, con perfiles o distancias reales, o con PCA.

▼ 5. Tu Isolation Forest marca el 5 % de las transacciones porque pusiste `contamination=0.05`. ¿Es un buen umbral?

Solo si coincide con lo que el equipo puede revisar y con la prevalencia esperada. `contamination` no se aprende de los datos: fija el umbral. Elige el umbral por capacidad de revisión y evalúa con casos etiquetados (PR-AUC, recall con falsas alarmas fijas).

▼ 6. Tu segmentación no usa el sexo del cliente. ¿Puedes afirmar que es neutral respecto del sexo?

No. Otras variables pueden funcionar como proxies. Mide la composición por sexo de cada segmento (razón de representación) y analiza el impacto según el uso previsto; para eso hay que conservar la variable sensible para auditar.

Lecturas de la semana

- **BB3** · James et al. (2023), ISLP: cap. 12 (Unsupervised Learning: PCA y clustering).
- **BB4** · Raschka et al. (2022): caps. 5 (reducción de dimensionalidad) y 10 (clustering).
- **BB1** · Géron (2025): capítulo de aprendizaje no supervisado (clustering, reducción de dimensionalidad y detección de anomalías).
- **BB5** · scikit-learn User Guide: Clustering, Decomposition (PCA), Manifold learning (t-SNE) y Novelty and outlier detection.
- **BB7** · Barocas, Hardt y Narayanan (2023), Fairness and Machine Learning: introducción y clasificación, como marco para el análisis por subgrupos.